



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ

Κάργας Στέφανος Διονύσης Κατσέτης
(el23059) (el23005)

Λειτουργικά Συστήματα Υπολογιστών
6ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

3η Εργαστηριακή Άσκηση:
Εικονική Μνήμη

Αθήνα 2026

Μέρος 1

1. Τυπώστε το χάρτη της εικονικής μνήμης της τρέχουσας διεργασίας.

Step 1: Print the virtual address space map of this process [1643886].

```
Virtual Memory Map of process [1643886]:
55bcc4c4000-55bcc4c4d000 r--p 00000000 00:26 9701861 /home/oslab/oslab018/lab3/mmap/mmap
55bcc4c4d000-55bcc4c4e000 r-xp 00001000 00:26 9701861 /home/oslab/oslab018/lab3/mmap/mmap
55bcc4c4e000-55bcc4c4f000 r--p 00002000 00:26 9701861 /home/oslab/oslab018/lab3/mmap/mmap
55bcc4c4f000-55bcc4c50000 r--p 00002000 00:26 9701861 /home/oslab/oslab018/lab3/mmap/mmap
55bcc4c50000-55bcc4c51000 rw-p 00003000 00:26 9701861 /home/oslab/oslab018/lab3/mmap/mmap
55bcc6331000-55bcc6352000 rw-p 00000000 00:00 0 [heap]
7f000f5fa000-7f000f61c000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f000f61c000-7f000f775000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f000f775000-7f000f7c4000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f000f7c4000-7f000f7c8000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f000f7c8000-7f000f7ca000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f000f7ca000-7f000f7d0000 rw-p 00000000 00:00 0
7f000f7d0000-7f000f7d7000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f000f7d7000-7f000f7f7000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f000f7f7000-7f000f7ff000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f000f800000-7f000f801000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f000f801000-7f000f802000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f000f802000-7f000f803000 rw-p 00000000 00:00 0
7ffff86f8000-7ffff8719000 rw-p 00000000 00:00 0 [stack]
7ffff872b000-7ffff872f000 r--p 00000000 00:00 0 [vvar]
7ffff872f000-7ffff8731000 r-xp 00000000 00:00 0 [vdso]
```

Εκτυπώσαμε τον χάρτη εικονικής μνήμης της διεργασίας μας με PID 1643886. Στο στιγμιότυπο φαίνεται ακριβώς πώς το λειτουργικό σύστημα έχει μοιράσει τη μνήμη για το πρόγραμμά μας. Μπορούμε να ξεχωρίσουμε εύκολα απο τα paths / pseudo paths το ίδιο το εκτελέσιμο αρχείο (mmap) στην αρχή, τη σωρό ([heap]) και τη στοίβα ([stack]) που χρησιμοποιεί το πρόγραμμα, καθώς και τις τον linker (ld) & βασικές βιβλιοθήκες του συστήματος όπως η libc που φορτώθηκαν στη μνήμη για να μπορέσει να τρέξει ο κώδικας.

2. Με την κλήση συστήματος mmap() δεσμεύστε buffer (προσωρινή μνήμη) μεγέθους μιας σελίδας (page) και τυπώστε ξανά το χάρτη. Εντοπίστε στον χάρτη μνήμης τον χώρο εικονικών διευθύνσεων που δεσμεύσατε.

Step 2: Use mmap(2) to allocate a private buffer of size equal to 1 page and print the VM map again.

```
Virtual Memory Map of process [1644170]:
56184868c000-56184868d000 r--p 00000000 00:26 9701863 /home/oslab/oslab018/lab3/mmap/mmap
56184868d000-56184868e000 r-xp 00001000 00:26 9701863 /home/oslab/oslab018/lab3/mmap/mmap
56184868e000-56184868f000 r--p 00002000 00:26 9701863 /home/oslab/oslab018/lab3/mmap/mmap
56184868f000-561848690000 r--p 00002000 00:26 9701863 /home/oslab/oslab018/lab3/mmap/mmap
561848690000-561848691000 rw-p 00003000 00:26 9701863 /home/oslab/oslab018/lab3/mmap/mmap
5618498e4000-561849905000 rw-p 00000000 00:00 0 [heap]
7f639560c000-7f639562e000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f639562e000-7f6395787000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f6395787000-7f63957d6000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f63957d6000-7f63957da000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f63957da000-7f63957dc000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f63957dc000-7f63957e2000 rw-p 00000000 00:00 0
7f63957e8000-7f63957e9000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f63957e9000-7f6395809000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f6395809000-7f6395811000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f6395811000-7f6395812000 rw-p 00000000 00:00 0
7f6395812000-7f6395813000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f6395813000-7f6395814000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f6395814000-7f6395815000 rw-p 00000000 00:00 0
7ffff0f18f000-7ffff0f1b000 rw-p 00000000 00:00 0 [stack]
7ffff0f1f9000-7ffff0f1fd000 r--p 00000000 00:00 0 [vvar]
7ffff0f1fd000-7ffff0f1ff000 r-xp 00000000 00:00 0 [vdso]
```

Στη συνέχεια της άσκησης, καλέσαμε την `mmap()` για να ζητήσουμε από το λειτουργικό μια σελίδα anonymous εικονικής μνήμης, με `rw-p` permissions. Παρατηρώντας τον νέο χάρτη, βλέπουμε ότι έχει προστεθεί μια νέα γραμμή ανάμεσα στις βιβλιοθήκες, συγκεκριμένα στο εύρος `7f6395811000-7f6395812000`. Καταλαβαίνουμε ότι αυτός είναι ο `buffer` που δεσμεύσαμε, αφού έχει δικαιώματα `rw-p` (ανάγνωση/εγγραφή & private mapping), κενό `path`, μηδενικό `major` & `minor` device number (for physical storage device), μηδενικό `offset` (for file-backed) και **inode (file id)** μηδενικό οπότε δεν αντιστοιχεί σε κάποιο αρχείο του δίσκου και είναι anonymous memory map.

3. Βρείτε και τυπώστε τη φυσική διεύθυνση μνήμης στην οποία απεικονίζεται η εικονική διεύθυνση του `buffer` (τη διεύθυνση όπου βρίσκεται αποθηκευμένος στη φυσική κύρια μνήμη). Τι παρατηρείτε και γιατί;

```
Step 3: Find and print the physical address of the buffer in main memory. What do you see?  
VA[0x7f2746e0b000] is not mapped; no physical memory allocated.  
Physical address of heap_private_buf: 0x0
```

Η έλλειψη αντιστοίχισης της εικονικής περιοχής με κάποια φυσική διεύθυνση είναι απολύτως αναμενόμενη, καθώς δεν έχει πραγματοποιηθεί ακόμα κάποια εγγραφή δεδομένων / δεν έχει γίνει “touch” το page από τη διεργασία. Το λειτουργικό σύστημα εφαρμόζει εδώ την τακτική του **demand paging**, αποφεύγοντας να σπαταλήσει φυσικούς πόρους στη RAM μέχρι τη στιγμή που η διεργασία θα τη προσπελάσει πραγματικά και θα κάνει **zero-fill**.

4. Γεμίστε με μηδενικά τον `buffer` και επαναλάβετε το Βήμα 3. Ποια αλλαγή παρατηρείτε;

```
Step 4: Initialize your buffer with zeros and repeat Step 3. What happened?  
Physical address of heap_private_buf after zero-fill: 0x1e7578000
```

Αρχικοποιήσαμε τον `buffer` με μηδενικά και δοκιμάσαμε να τυπώσουμε ξανά τις φυσικές διευθύνσεις. Το αποτέλεσμα είναι αναμενόμενα διαφορετικό. Καθώς το πρόγραμμα **αγγίζει** για πρώτη φορά το page για να γράψει, ενεργοποιείται ο μηχανισμός του demand paging και δεσμεύεται ένα physical frame σε μια ελεύθερη θέση της main. Το λειτουργικό σύστημα ανταποκρίνεται άμεσα και συνδέει την virtual address με αυτό μέσω entry στο Page Table και έτσι εμφανίζεται η φυσική διεύθυνση που απεικονίστηκε.

5. Χρησιμοποιείτε την `mmap()` για να απεικονίσετε (memory map) το αρχείο `file.txt` στον χώρο διευθύνσεων της διεργασίας σας και να τυπώσετε το περιεχόμενό του. Εντοπίστε τη νέα απεικόνιση (mapping) στον χάρτη μνήμης.

Με τη χρήση της `mmap()` διαβάζουμε και απεικονίζουμε το `file.txt` στον χώρο διευθύνσεων της διεργασίας. Ύστερα, τυπώνουμε στο τερματικό το περιεχόμενο του αρχείου και τον ανανεωμένο χάρτη μνήμης της διεργασίας:

```
Step 5: Use mmap(2) to read and print file.txt. Print the new mapping information that has been created.

Content of file.txt: Hello everyone!

Virtual Memory Map of process [1647594]:
55e589845000-55e589846000 r--p 00000000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
55e589846000-55e589847000 r-xp 00001000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
55e589847000-55e589848000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
55e589848000-55e589849000 r--p 00003000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
55e589849000-55e58984a000 rw-p 00004000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
55e589c52000-55e589c73000 rw-p 00000000 00:00 0 [heap]
7f470ceff000-7f470cf21000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f470cf21000-7f470d07a000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f470d07a000-7f470d0c9000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f470d0c9000-7f470d0cd000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f470d0cd000-7f470d0cf000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f470d0cf000-7f470d0d5000 rw-p 00000000 00:00 0
7f470d0d5000-7f470d0db000 r--p 00000000 00:26 9701723 /home/oslab/oslab018/lab3/mmap/file.txt
7f470d0db000-7f470d0dc000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f470d0dc000-7f470d0fc000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f470d0fc000-7f470d104000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f470d104000-7f470d105000 rw-p 00000000 00:00 0
7f470d105000-7f470d106000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f470d106000-7f470d107000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f470d107000-7f470d108000 rw-p 00000000 00:00 0
7fff65a30000-7fff65a51000 rw-p 00000000 00:00 0 [stack]
7fff65ae2000-7fff65ae6000 r--p 00000000 00:00 0 [vvar]
7fff65ae6000-7fff65ae8000 r-xp 00000000 00:00 0 [vdso]
```

Παρατηρούμε από το path το `/home/oslab/oslab018/lab3/mmap/file.txt` την απεικόνιση του `file.txt` στον vma space. Πιο συγκεκριμένα, επειδή υλοποιήσαμε file mapping ήταν παρατηρούμε τα πεδία `major:minor device number = 00:26 != 00:00` (λόγω αποθήκευσης του σε virtual file system αφού τα αρχεία, εξαιτίας της δομής των `node{1,2}` χρησιμοποιούν Network File System-NFS, οπότε λογικό το `major = 00` που υποδηλώνει απουσία φυσικής αποθήκευσης σε δίσκο), `offset = 00000000` (αφού στο operand της `mmap` βάλαμε `offset 0`), `inode number (file id) = 9701723 != 0` (0 θα παίρναμε μόνο για ανώνυμη απεικόνιση, `file.txt` backed in disk), με δικαιώματα `r-p` αφού στην `open` βάλαμε μόνο `O_RDONLY` και στο `mmap` protection `PROT_READ` και flag `MAP_PRIVATE`.

6. Χρησιμοποιείστε την `mmap()` για να δεσμεύσετε έναν νέο buffer, διαμοιραζόμενο (shared) αυτή τη φορά μεταξύ διεργασιών με μέγεθος μιας σελίδας. Εντοπίστε τη νέα απεικόνιση (mapping) στο χάρτη μνήμης.

Με τη `mmap()` δεσμεύουμε νέο buffer μεγέθους μιας σελίδας, ο οποίος είναι `shared` μεταξύ των διεργασιών:

Step 6: Use mmap(2) to allocate a shared buffer of size equal to 1 page. Initialize the buffer and print the new mapping information that has been created.

Shared buffer starts at virtual address: 0x7f06412c4000

```
Virtual Memory Map of process [1647783]:
556593a17000-556593a18000 r--p 00000000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556593a18000-556593a19000 r-xp 00001000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556593a19000-556593a1a000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556593a1a000-556593a1b000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556593a1b000-556593a1c000 rw-p 00003000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556594187000-5565941a8000 rw-p 00000000 00:00 0 [heap]
7f06410ea000-7f064110c000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f064110c000-7f0641265000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f0641265000-7f06412b4000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f06412b4000-7f06412b8000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f06412b8000-7f06412ba000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f06412ba000-7f06412c0000 rw-p 00000000 00:00 0
7f06412c4000-7f06412c5000 rw-s 00000000 00:01 18553 /dev/zero (deleted)
7f06412c5000-7f06412c6000 r--p 00000000 00:26 9701723 /home/oslab/oslab018/lab3/mmap/file.txt
7f06412c6000-7f06412c7000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f06412c7000-7f06412e7000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f06412e7000-7f06412ef000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f06412ef000-7f06412f0000 rw-p 00000000 00:00 0
7f06412f0000-7f06412f1000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f06412f1000-7f06412f2000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f06412f2000-7f06412f3000 rw-p 00000000 00:00 0
7ffc2692b000-7ffc2694c000 rw-p 00000000 00:00 0 [stack]
7ffc26959000-7ffc2695d000 r--p 00000000 00:00 0 [vvar]
7ffc2695d000-7ffc2695f000 r-xp 00000000 00:00 0 [vdso]
```

Ο shared buffer εντοπίζεται στον χάρτη μνήμης από τα flag **-s** (shared) (πέρα από τα permissions **rw** που ορθώς εκχωρήθηκαν αφού καλέσαμε την **mmap** με **PROT_READ**, **PROT_WRITE** - **MAP_SHARED** και αποτελείται από τις εικονικές διευθύνσεις **7f06412c4000 - 7f06412c5000**.

Παρατηρούμε ότι στο path του **mmap** για αυτές τις **vma**, είναι το **/dev/zero/** (deleted). Αυτό συμβαίνει, καθώς το mapping με **shared** flag μπορεί να πραγματοποιηθεί μόνο αν υπάρχει ένα **shared** object, όπως κάνει η **shm_open** (POSIX) με το OS να το σώζει στο **tmfs**. Η διαφορά εδώ είναι πως μόλις γίνει το mapping διαγράφεται αυτό το object (deleted) αλλά το **inode** κρατιέται (18553) όσο είναι το mapping υποστηριζόμενο από την διεργασία.

Επειδή αυτό το αντικείμενο πλέον έχει διαγραφεί και η μνήμη που έγινε mapped είναι ανώνυμη, ο μοναδικός τρόπος να υλοποιήσουμε IPC με **shared** memory μηχανισμό είναι με την syscall **fork()** όπου το παιδί κληρονομεί το Page Table (copy) του γονέα και τα entries του δείχνουν στο ίδιο frame της **main**. Το mapping είναι **shared** οπότε ο μηχανισμός **cow** δεν πραγματοποιείται, και έτσι πετυχαίνουμε την συγκεκριμένη επικοινωνία, όπως θα δούμε στα παρακάτω ερωτήματα.

7. Τυπώστε τους χάρτες της εικονικής μνήμης της γονικής διεργασίας και της διεργασίας παιδιού. Τι παρατηρείτε?

Step 7: Print parent's and child's map.

[PARENT] Virtual Memory Map of Parent process (PID: 1647915):

```
Virtual Memory Map of process [1647915]:
561be6d17000-561be6d18000 r--p 00000000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d18000-561be6d19000 r-xp 00001000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d19000-561be6d1a000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d1a000-561be6d1b000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d1b000-561be6d1c000 rw-p 00003000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6e38000-561be6e59000 rw-p 00000000 00:00 0 [heap]
7f2eb3d6b000-7f2eb3d8d000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3d8d000-7f2eb3ee6000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3ee6000-7f2eb3f35000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3f35000-7f2eb3f39000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3f39000-7f2eb3f3b000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3f3b000-7f2eb3f41000 rw-p 00000000 00:00 0
7f2eb3f45000-7f2eb3f46000 rw-s 00000000 00:01 17552 /dev/zero (deleted)
7f2eb3f46000-7f2eb3f47000 r--p 00000000 00:26 9701723 /home/oslab/oslab018/lab3/mmap/file.txt
7f2eb3f47000-7f2eb3f48000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f48000-7f2eb3f68000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f68000-7f2eb3f70000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f70000-7f2eb3f71000 rw-p 00000000 00:00 0
7f2eb3f71000-7f2eb3f72000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f72000-7f2eb3f73000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f73000-7f2eb3f74000 rw-p 00000000 00:00 0
7ffd245bb000-7ffd245dc000 rw-p 00000000 00:00 0 [stack]
7ffd245e8000-7ffd245ec000 r--p 00000000 00:00 0 [vvar]
7ffd245ec000-7ffd245ee000 r-xp 00000000 00:00 0 [vdso]
```

[CHILD] Virtual Memory Map of Child process (PID: 1647916):

```
Virtual Memory Map of process [1647916]:
561be6d17000-561be6d18000 r--p 00000000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d18000-561be6d19000 r-xp 00001000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d19000-561be6d1a000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d1a000-561be6d1b000 r--p 00002000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6d1b000-561be6d1c000 rw-p 00003000 00:26 9701721 /home/oslab/oslab018/lab3/mmap/mmap
561be6e38000-561be6e59000 rw-p 00000000 00:00 0 [heap]
7f2eb3d6b000-7f2eb3d8d000 rw-p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3d8d000-7f2eb3ee6000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3ee6000-7f2eb3f35000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3f35000-7f2eb3f39000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3f39000-7f2eb3f3b000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2eb3f3b000-7f2eb3f41000 rw-p 00000000 00:00 0
7f2eb3f45000-7f2eb3f46000 rw-s 00000000 00:01 17552 /dev/zero (deleted)
7f2eb3f46000-7f2eb3f47000 r--p 00000000 00:26 9701723 /home/oslab/oslab018/lab3/mmap/file.txt
7f2eb3f47000-7f2eb3f48000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f48000-7f2eb3f68000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f68000-7f2eb3f70000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f70000-7f2eb3f71000 rw-p 00000000 00:00 0
7f2eb3f71000-7f2eb3f72000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f72000-7f2eb3f73000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2eb3f73000-7f2eb3f74000 rw-p 00000000 00:00 0
7ffd245bb000-7ffd245dc000 rw-p 00000000 00:00 0 [stack]
7ffd245e8000-7ffd245ec000 r--p 00000000 00:00 0 [vvar]
7ffd245ec000-7ffd245ee000 r-xp 00000000 00:00 0 [vdso]
```

Παρατηρούμε ότι οι δύο χώροι διευθύνσεων είναι δομικά πανομοιότυποι. Πιο συγκεκριμένα, επιβεβαιώνεται η λειτουργία της κλήσης συστήματος `fork()`, μέσω της οποίας το OS δημιουργεί ένα ακριβές αντίγραφο του Page Table του γονέα. Εστιάζοντας στη **διαμοιραζόμενη περιοχή**, βλέπουμε πως και οι δύο διεργασίες την απεικονίζουν στο ίδιο ακριβές εύρος διευθύνσεων με **κοινό inode** και δικαιώματα `rw-s`.

Η παρουσία του flag `s` (shared) εξασφαλίζει ότι οι αντίστοιχες εγγραφές των πινάκων σελίδων μεταφράζονται στο ίδιο frame μνήμης, επιτρέποντας IPC με μηχανισμό **shared memory**, όπως και προβλέψαμε στο παραπάνω ερώτημα.

Έτσι, σε αυτήν τη συγκεκριμένη διαμοιραζόμενη περιοχή ο μηχανισμός **cow** **παρακάμπτεται**. Αντιθέτως, στις υπόλοιπες εγγράψιμες περιοχές, όπως ο σωρός και η στοίβα, διατηρούνται αυστηρά τα δικαιώματα ιδιωτικότητας `rw-p`. Για κάθε private (non-shared) page, το OS προστατεύει τη φυσική μνήμη επιβάλλοντας τον μηχανισμό **Copy-on-Write**, εκχωρώντας νέα frames μόνο όταν κάποια από τις διεργασίες επιχειρήσει να γράψει στα δεδομένα τους, αφού ακολουθήσει πρώτα ένα Page Fault.

8. Βρείτε και τυπώστε τη φυσική διεύθυνση στη κύρια μνήμη του private buffer (Βήμα 3) για τις διεργασίες γονέα και παιδιού. Τι συμβαίνει αμέσως μετά το fork?

```
Step 8: Find the physical address of the private heap buffer (main) for both the parent and the child.  
[PARENT] Physical address of heap_private_buf: 0x1b07e6000  
[CHILD] Physical address of heap_private_buf: 0x1b07e6000
```

Παρατηρούμε πως οι 2 φυσικές διευθύνσεις ταυτίζονται. Αυτό συμβαίνει καθώς, μετά το `fork()`, δεν έχει γίνει κάποια προσπάθεια ακόμα για `write` από γονέα ή παιδί οπότε το Copy-on-Write εγγυάται ότι θα γίνει αντιγραφή φυσικής μνήμης μόνο όταν μια ενέργεια εγγραφής πραγματοποιηθεί σε κάποιο page μη διαμοιραζόμενο. Εδώ το private buffer είναι ιδιωτικό page, οπότε επιβεβαιώνεται το αποτέλεσμα.

9. Γράψτε στον private buffer από τη διεργασία παιδί και επαναλάβετε το Βήμα 8. Τι αλλάζει και γιατί?

```
Step 8: Find the physical address of the private heap buffer (main) for both the parent and the child.  
[PARENT] Physical address of heap_private_buf: 0x158364000  
[CHILD] Physical address of heap_private_buf: 0x158364000  
  
Step 9: Write to the private buffer from the child and repeat step 8. What happened?  
[CHILD] Physical address of heap_private_buf after write: 0x24b2c2000  
[PARENT] Physical address of heap_private_buf: 0x158364000
```

**Οι physical addresses είναι διαφορετικές από το βήμα 8 αφού ξανατρέχουμε το πρόγραμμα.*

Μετά την εγγραφή δεδομένων από τη διεργασία - παιδί στον εν λόγω buffer, διαπιστώνεται ότι η φυσική της διεύθυνση μεταβάλλεται στο νέο frame, ενώ η αντίστοιχη διεύθυνση της γονικής διεργασίας παραμένει αμετάβλητη. Η συγκεκριμένη απόπειρα εγγραφής προκάλεσε **Page Fault (λόγω cow)**, αναγκάζοντας τον kernel να τερματίσει τον διαμοιρασμό. Το OS δέσμευσε ένα νέο frame φυσικής μνήμης για το

παιδί, αντέγραψε τα αρχικά δεδομένα και ενημέρωσε τον αντίστοιχο Page Table.

10. Γράψτε στον shared buffer (Βήμα 6) από τη διεργασία παιδί και τυπώστε τη φυσική του διεύθυνση για τις διεργασίες γονέα και παιδιού. Τι παρατηρείτε σε σύγκριση με τον private buffer?

```
Step 10: Write to the shared heap buffer (main) from child and get the physical address for both the parent and the child. What happened?

[CHILD] Physical address of heap_shared_buf after write: 0x1716af000
[PARENT] Physical address of heap_shared_buf: 0x1716af000
[PARENT] Data read from shared buffer: Shared data from child!
```

Παρατηρούμε πως σε σύγκριση με τον ιδιωτικό buffer, εδώ το mapping είναι shared (shared buffer) οπότε ο μηχανισμός cow παρακάμπτεται και πετυχαίνουμε IPC με shared memory επιτυχώς. Ο Page Table λόγω κληρονομικότητας στο παιδί, πλέον δείχνει στο ίδιο φυσικό frame για αυτή την διαμοιραζόμενη περιοχή μνήμης, χωρίς το OS να “κατεβάσει” τα δικαιώματα εγγραφής για τις 2 διεργασίες, αναγκάζοντας τοπικές αλλαγές και μόνο. Έτσι, ότι γράφει το παιδί/γονέας γίνεται ορατό από την άλλη διεργασία και δεν δημιουργείται κανένα Page Fault εξ αρχής κατά τη πρώτη προσπάθεια εγγραφής.

11. Απαγορεύστε τις εγγραφές στον shared buffer για τη διεργασία παιδί. Εντοπίστε και τυπώστε την απεικόνιση του shared buffer στο χάρτη μνήμης των δύο διεργασιών για να επιβεβαιώσετε την απαγόρευση.

```
[CHILD] Virtual Memory Map of Child process AFTER mprotect (PID: 1062938):

Virtual Memory Map of process [1062938]:
556bc2486000-556bc2487000 r--p 00000000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2487000-556bc2488000 r-xp 00001000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2488000-556bc2489000 r--p 00002000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2489000-556bc248a000 r--p 00002000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc248a000-556bc248b000 rw-p 00003000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2e7a000-556bc2e7b000 rw-p 00000000 00:00 0 [heap]
7feb9e990000-7feb9e99b000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9e99b000-7feb9e99d000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9e99d000-7feb9e99e000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9e99e000-7feb9e99f000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9e99f000-7feb9e9a0000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9e9a0000-7feb9e9a1000 rw-p 00000000 00:00 0
7feb9e9a1000-7feb9e9a2000 r--s 00000000 00:01 795 /dev/zero (deleted)
7feb9e9a2000-7feb9e9a3000 r--p 00000000 00:27 9701723 /home/oslab/oslab018/lab3/mmap/file.txt
7feb9e9a3000-7feb9e9a4000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9e9a4000-7feb9e9a5000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9e9a5000-7feb9e9a6000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9e9a6000-7feb9e9a7000 rw-p 00000000 00:00 0
7feb9e9a7000-7feb9e9a8000 rw-p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9e9a8000-7feb9e9a9000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9e9a9000-7feb9e9aa000 rw-p 00000000 00:00 0
7fff5e8db000-7fff5e8fc000 rw-p 00000000 00:00 0 [stack]
7fff5e92c000-7fff5e930000 r--p 00000000 00:00 0 [vvar]
7fff5e930000-7fff5e932000 r-xp 00000000 00:00 0 [vdso]
```

```

[PARENT] Virtual Memory Map of Parent process (PID: 1062937):
Virtual Memory Map of process [1062937]:
556bc2486000-556bc2487000 r--p 00000000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2487000-556bc2488000 r-xp 00001000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2488000-556bc2489000 r--p 00002000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2489000-556bc248a000 r--p 00002000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc248a000-556bc248b000 rw-p 00003000 00:27 9701721 /home/oslab/oslab018/lab3/mmap/mmap
556bc2e7a000-556bc2e9b000 rw-p 00000000 00:00 0 [heap]
7feb9e990000-7feb9eb2000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9eb2000-7feb9eb0b000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9eb0b000-7feb9eb5a000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9eb5a000-7feb9eb5e000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9eb5e000-7feb9eb60000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7feb9eb60000-7feb9eb60000 rw-p 00000000 00:00 0
7feb9eb6a000-7feb9eb6b000 rw-s 00000000 00:01 795 /dev/zero (deleted)
7feb9eb6b000-7feb9eb6c000 r--p 00000000 00:27 9701723 /home/oslab/oslab018/lab3/mmap/file.txt
7feb9eb6c000-7feb9eb6d000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9eb6d000-7feb9eb80000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9eb80000-7feb9eb95000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9eb95000-7feb9eb96000 rw-p 00000000 00:00 0
7feb9eb96000-7feb9eb97000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9eb97000-7feb9eb98000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7feb9eb98000-7feb9eb99000 rw-p 00000000 00:00 0
7fff5e8db000-7fff5e8fc000 rw-p 00000000 00:00 0 [stack]
7fff5e92c000-7fff5e930000 r--p 00000000 00:00 0 [vvar]
7fff5e930000-7fff5e932000 r-xp 00000000 00:00 0 [vdso]
-----

```

Από την αντιπαραβολή των χαρτών εικονικής μνήμης των δύο διεργασιών, επιβεβαιώνεται πρακτικά η επιτυχής τροποποίηση των δικαιωμάτων πρόσβασης. Πιο συγκεκριμένα, στην vma του διαμοιραζόμενου buffer (διευθύνσεις 7feb9eb6a000-7feb9eb6b000), παρατηρούμε ότι τα δικαιώματα της διεργασίας - παιδιού έχουν μεταβληθεί σε **r-s**, ενώ η γονική διεργασία διατηρεί ανέπαφα τα αρχικά δικαιώματα **rw-s**.

Έτσι, καταλαβαίνουμε ότι κλήση συστήματος `mprotect()` επιδρά τοπικά και αποκλειστικά στον Memory Map της διεργασίας που την καλεί. Παράλληλα, η διατήρηση της σημαίας `s` (shared) και του κοινού inode (795) αποδεικνύει ότι η εικονική αυτή περιοχή και των δύο διεργασιών εξακολουθεί να χαρτογραφείται στο ίδιο κοινό frame φυσικής μνήμης.

12. Αποδεσμεύστε όλους τους buffers στις δύο διεργασίες.

Step 12: Free all buffers for parent and child.

```

[PARENT] All buffers unmapped successfully. Exiting.
oslab018@os-node1:~/lab3/mmap$

```

Εκτελέσαμε τη κλήση συστήματος **`munmap()`** και από τις δύο διεργασίες για την αποδέσμευση των vma που είχαν δεσμευτεί στα προηγούμενα βήματα. Με την κλήση αυτή, οι εγγραφές στους Page Tables ακυρώνονται και η MMU απελευθερώνει τα αντίστοιχα φυσικά frames στη RAM.

Source code (.c):

```
C/C++
/*
 * mmap.c
 *
 * Examining the virtual memory of processes.
 *
 * Operating Systems course, CSLab, ECE, NTUA
 */
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <errno.h>
#include <stdint.h>
#include <signal.h>
#include <sys/wait.h>
#include "help.h"
#define RED      "\033[31m"
#define RESET   "\033[0m"
char *heap_private_buf;
char *heap_shared_buf;
char *file_shared_buf;
uint64_t buffer_size;
/*
 * Child process' entry point.
 */
void child(void)
{
    uint64_t pa;
    /*
     * Step 7 - Child
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    /*
     * Step 7.
     */
    printf("\n[CHILD] Virtual Memory Map of Child process (PID: %d):\n", getpid());
    show_maps();
    /*
     * Step 8 - Child
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    /*
     * Step 8.
     */
    uint64_t child_pa = get_physical_address((uint64_t)heap_private_buf);
    printf("[CHILD] Physical address of heap_private_buf: 0x%llx\n", (unsigned
long long)child_pa);
    /*
     * Step 9 - Child
     */
    if (0 != raise(SIGSTOP))
```

```

        die("raise(SIGSTOP)");
    /*
     * Step 9.
     */
    strcpy(heap_private_buf, "I am the child and i am writting!");
    uint64_t child_pa_after_write =
    get_physical_address((uint64_t)heap_private_buf);
    printf("[CHILD] Physical address of heap_private_buf after write:
    0x%llx\n", (unsigned long long)child_pa_after_write);
    /*
     * Step 10 - Child
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    /*
     * Step 10.
     */
    strcpy(heap_shared_buf, "Shared data from child!");
    uint64_t child_shared_pa = get_physical_address((uint64_t)heap_shared_buf);
    printf("[CHILD] Physical address of heap_shared_buf after write:
    0x%llx\n", (unsigned long long)child_shared_pa);
    /*
     * Step 11 - Child
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    /*
     * Step 11.
     */
    if(mprotect(heap_shared_buf,buffer_size,PROT_READ)==-1) die("mprotect child");
    printf("\n[CHILD] Virtual Memory Map of Child process AFTER mprotect (PID:
    %d):\n", getpid());
    show_maps();
    /*
     * Step 12 - Child
     */
    /*
     * Step 12.
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    if (munmap(heap_private_buf, buffer_size)==-1) die("munmap heap_private_buf
    child");
    if (munmap(heap_shared_buf, buffer_size)==-1) die("munmap heap_shared_buf
    child");
    if (munmap(file_shared_buf, buffer_size)==-1) die("munmap file_shared_buf
    child");
}
/*
 * Parent process' entry point.
 */
void parent(pid_t child_pid)
{
    uint64_t pa;
    int status;
    /* Wait for the child to raise its first SIGSTOP. */
    if (-1 == waitpid(child_pid, &status, WUNTRACED))
        die("waitpid");
    /*
     * Step 7: Print parent's and child's maps. What do you see?
     * Step 7 - Parent

```

```

    */
    printf(RED "\nStep 7: Print parent's and child's map.\n" RESET);
    press_enter();
    /*
    *Step 7.
    */
    printf("\n[PARENT] Virtual Memory Map of Parent process (PID: %d):\n",
getpid());
    show_maps();
    if (-1 == kill(child_pid, SIGCONT))
        die("kill");
    if (-1 == waitpid(child_pid, &status, WUNTRACED))
        die("waitpid");
    /*
    * Step 8: Get the physical memory address for heap_private_buf.
    * Step 8 - Parent
    */
    printf(RED "\nStep 8: Find the physical address of the private heap "
        "buffer (main) for both the parent and the child.\n" RESET);
    press_enter();
    /*
    * Step 8.
    */
    uint64_t parent_pa = get_physical_address((uint64_t)heap_private_buf);
    printf("[PARENT] Physical address of heap_private_buf: 0x%llx\n", (unsigned
long long)parent_pa);
    if (-1 == kill(child_pid, SIGCONT))
        die("kill");
    if (-1 == waitpid(child_pid, &status, WUNTRACED))
        die("waitpid");
    /*
    * Step 9: Write to heap_private_buf. What happened?
    * Step 9 - Parent
    */
    printf(RED "\nStep 9: Write to the private buffer from the child and "
        "repeat step 8. What happened?\n" RESET);
    press_enter();
    /*
    * Step 9.
    */
    if (-1 == kill(child_pid, SIGCONT))
        die("kill");
    if (-1 == waitpid(child_pid, &status, WUNTRACED))
        die("waitpid");
    parent_pa = get_physical_address((uint64_t)heap_private_buf);
    printf("[PARENT] Physical address of heap_private_buf: 0x%llx\n", (unsigned
long long)parent_pa);
    /*
    * Step 10: Get the physical memory address for heap_shared_buf.
    * Step 10 - Parent
    */
    printf(RED "\nStep 10: Write to the shared heap buffer (main) from "
        "child and get the physical address for both the parent and "
        "the child. What happened?\n" RESET);
    press_enter();
    /*
    * Step 10.
    */
    if (-1 == kill(child_pid, SIGCONT))
        die("kill");
    if (-1 == waitpid(child_pid, &status, WUNTRACED))

```



```

        die("waitpid");
uint64_t parent_shared_pa = get_physical_address((uint64_t)heap_shared_buf);
printf("[PARENT] Physical address of heap_shared_buf: 0x%llx\n", (unsigned
long long)parent_shared_pa);
printf("[PARENT] Data read from shared buffer: %s\n", (char *)heap_shared_buf);
/*
 * Step 11: Disable writing on the shared buffer for the child
 * (hint: mprotect(2)).
 * Step 11 - Parent
 */
printf(RED "\nStep 11: Disable writing on the shared buffer for the "
        "child. Verify through the maps for the parent and the "
        "child.\n" RESET);
press_enter();
/*
 * Step 11
 */
if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");
printf("\n[PARENT] Virtual Memory Map of Parent process (PID: %d):\n",
getpid());
show_maps();
/*
 * Step 12: Free all buffers for parent and child.
 * Step 12 - Parent
 */
/*
 * Step 12.
 */
printf(RED "\nStep 12: Free all buffers for parent and child.\n" RESET);
press_enter();
if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, 0))
    die("waitpid");
if (munmap(heap_private_buf, buffer_size)==-1) die("munmap heap_private_buf
parent");
if (munmap(heap_shared_buf, buffer_size)==-1) die("munmap heap_shared_buf
parent");
if (munmap(file_shared_buf, buffer_size)==-1) die("munmap file_shared_buf
parent");
printf("[PARENT] All buffers unmapped successfully. Exiting.\n");
}
int main(void)
{
    pid_t mypid, p;
    int fd = -1;
    uint64_t pa;
    mypid = getpid();
    buffer_size = 1 * get_page_size();
    /*
     * Step 1: Print the virtual address space layout of this process.
     */
    printf(RED "\nStep 1: Print the virtual address space map of this "
            "process [%d].\n" RESET, mypid); press_enter();
    /* * Step 1. */
    show_maps();
    /*
     * Step 2: Use mmap to allocate a buffer of 1 page and print the map

```

```

        * again. Store buffer in heap_private_buf.
        */
printf(RED "\nStep 2: Use mmap(2) to allocate a private buffer of "
        "size equal to 1 page and print the VM map again.\n" RESET);
press_enter();
/*
 * Step 2.
 */
heap_private_buf =
mmap(NULL,buffer_size,PROT_READ|PROT_WRITE,MAP_ANONYMOUS|MAP_PRIVATE,-1,0);
if (heap_private_buf == MAP_FAILED) {
    die("mmap private anonymous");
}
show_maps();
/*
 * Step 3: Find the physical address of the first page of your buffer
 * in main memory. What do you see?
 */
printf(RED "\nStep 3: Find and print the physical address of the "
        "buffer in main memory. What do you see?\n" RESET);
press_enter();
/*
 * Step 3.
 */
pa = get_physical_address((uint64_t)heap_private_buf);
printf("Physical address of heap_private_buf: 0x%llx\n", (unsigned long
long)pa);
/*
 * Step 4: Write zeros to the buffer and repeat Step 3.
 */
printf(RED "\nStep 4: Initialize your buffer with zeros and repeat "
        "Step 3. What happened?\n" RESET);
press_enter();
/*
 * Step 4
 */
memset(heap_private_buf, 0, buffer_size);
pa = get_physical_address((uint64_t)heap_private_buf);
printf("Physical address of heap_private_buf after zero-fill: 0x%llx\n",
(unsigned long long)pa);
/*
 * Step 5: Use mmap(2) to map file.txt (memory-mapped files) and print
 * its content. Use file_shared_buf.
 */
printf(RED "\nStep 5: Use mmap(2) to read and print file.txt. Print "
        "the new mapping information that has been created.\n" RESET);
press_enter();
/*
 * Step 5.
 */
fd = open("file.txt", O_RDONLY);
if (fd == -1) {
    die("open file.txt");
}
file_shared_buf = mmap(NULL, buffer_size, PROT_READ, MAP_PRIVATE, fd, 0);
if (file_shared_buf == MAP_FAILED) {
    die("mmap file.txt");
}
printf("Content of file.txt: %s\n", file_shared_buf);
show_maps();
/*

```

```

    * Step 6: Use mmap(2) to allocate a shared buffer of 1 page. Use
    * heap_shared_buf.
    */
    printf(RED "\nStep 6: Use mmap(2) to allocate a shared buffer of size "
           "equal to 1 page. Initialize the buffer and print the new "
           "mapping information that has been created.\n" RESET);
    press_enter();
    /*
    * Step 6.
    */
    heap_shared_buf =
    mmap(NULL, buffer_size, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
    if(heap_shared_buf == MAP_FAILED) die("mmap shared obj");
    memset(heap_shared_buf, 0, buffer_size);
    printf("Shared buffer starts at virtual address: %p\n",
    (void*)heap_shared_buf);
    show_maps();
    p = fork();
    if (p < 0)
        die("fork");
    if (p == 0) {
        child();
        return 0;
    }
    parent(p);
    if (-1 == close(fd))
        perror("close");
    return 0;
}

```

Μέρος 2:

Semaphores πάνω από διαμοιραζόμενη μνήμη.

1. Ποια από τις δύο παραλληλοποιημένες υλοποιήσεις (threads vs processes) περιμένετε να έχει καλύτερη επίδοση και γιατί; Πώς επηρεάζει την επίδοση της υλοποίησης με διεργασίες το γεγονός ότι τα semaphores βρίσκονται σε διαμοιραζόμενη μνήμη μεταξύ διεργασιών;

Μεταξύ των δύο παραλληλοποιημένων υλοποιήσεων, αναμένεται εμπειρικά ότι η υλοποίηση με νήματα θα παρουσιάσει αισθητά καλύτερη επίδοση έναντι της υλοποίησης με διεργασίες. Πιο συγκεκριμένα, αυτό συμβαίνει διότι τα νήματα μοιράζονται εξ ορισμού τη μνήμη (εικονική και φυσική) και τους πόρους της διεργασίας στην οποία ανήκουν, καθιστώντας τόσο τη δημιουργία τους όσο και την εναλλαγή περιβάλλοντος πολύ πιο οικονομικές και ταχύτερες λειτουργίες συγκριτικά με τη δημιουργία εντελώς νέων διεργασιών.

Στην υλοποίηση με διεργασίες, παρόλο που η τοποθέτηση των ανώνυμων POSIX σηματοφόρων σε μια περιοχή διαμοιραζόμενης μνήμης εξασφαλίζει ταχύτητα

διαδιεργασιακή επικοινωνία (IPC), δεν εξαλείφει την εγγενή καθυστέρηση του ίδιου του μηχανισμού συγχρονισμού. Εάν μια διεργασία-παιδί εκτελέσει τη συνάρτηση `sem_wait()` και ο σημαφόρος δεν είναι διαθέσιμος, το λειτουργικό σύστημα αναγκάζεται να αναστείλει τη διεργασία, θέτοντάς την σε κατάσταση ύπνου. Αυτή η παύση υποχρεώνει τον χρονοδρομολογητή να αναλάβει τον έλεγχο και να πραγματοποιήσει μια εξ ολοκλήρου νέα αλλαγή περιβάλλοντος λειτουργίας, μια διαδικασία που απαιτεί χιλιάδες κύκλους ρολογιού και αποτελεί καθαρό χαμένο χρόνο για την CPU.

Το context switch των νημάτων είναι σαφώς λιγότερο χρονοβόρο από αυτό των διεργασιών. Ειδικότερα, για τις διεργασίες, ο πυρήνας χρειάζεται να εναλλάξει ολόκληρο τον Page Table της διεργασίας που βγαίνει και να βάλει αυτόν της διεργασίας που μπαίνει (που είναι ο ίδιος) και έτσι οφείλει να ενημερώσει τους καταχωρητές διαχείρισης μνήμης της Κεντρικής Μονάδας Επεξεργασίας, ώστε να δείχνουν στον εντελώς νέο πίνακα σελίδων της εισερχόμενης διεργασίας. Η ενέργεια αυτή προκαλεί αναπόφευκτα TLB flush του. Δεδομένου ότι ο TLB είναι πλέον άδειος, η νέα διεργασία θα υποστεί ένα μαζικό κύμα TLB misses και θα εξαναγκαστεί σε εξαιρετικά αργές διασχίσεις του πίνακα σελίδων (**page-table walks**) στην κύρια μνήμη, μέχρις ότου η κρυφή μνήμη επαναπληρωθεί με τις έγκυρες μεταφράσεις διευθύνσεων. Αντιθέτως, τα νήματα μοιράζονται τον ίδιο Page Table και γλυτώνουν αυτό το μεγάλο overhead της εναλλαγής και των αστοχιών του TLB.

Επομένως, το βαρύ κόστος των συνεχών εναλλαγών πλαισίου στον πυρήνα υποβαθμίζει τελικά την επίδοση της υλοποίησης με διεργασίες.

2. Μπορεί το mmap() interface να χρησιμοποιηθεί για τον διαμοιρασμό μνήμης μεταξύ διεργασιών που δεν έχουν κοινό ancestor; Αν όχι, γιατί;

Η διεπαφή της κλήσης συστήματος `mmap()` δύναται απολύτως να χρησιμοποιηθεί για τον διαμοιρασμό μνήμης μεταξύ διεργασιών που δεν έχουν κοινό πρόγονο, ωστόσο αυτό **αποκλείεται** εάν χρησιμοποιηθεί η σημαία της *ανώνυμης* απεικόνισης (**MAP_ANONYMOUS**).

Επειδή η ανώνυμη μνήμη δεν συσχετίζεται με κανένα αναγνωριστικό όνομα ή διαδρομή στο Εικονικό Σύστημα Αρχείων (VFS e.g., `tmfs`), παραμένει πλήρως μη εντοπίσιμη από εντελώς ανεξάρτητες διεργασίες. Κατά συνέπεια, η πρόσβαση σε **ανώνυμα** διαμοιραζόμενα αντικείμενα προϋποθέτει ρητά την κληρονόμηση των hardware page tables από έναν κοινό δημιουργό μέσω της κλήσης **fork()**.

Για να επιτευχθεί διαμοιρασμός μνήμης χωρίς κοινό πρόγονο, απαιτείται αρχιτεκτονικά η εγκαθίδρυση μιας απεικόνισης που υποστηρίζεται από αρχείο (**file-backed mapping**) ή η δημιουργία ενός επώνυμου αντικειμένου διαμοιραζόμενης μνήμης POSIX. Μέσω κλήσεων συστήματος όπως η `open()` ή η **`shm_open()`**, οι ανεξάρτητες διεργασίες μπορούν να αναφερθούν στο ίδιο κοινό συμβολοσειριακό

όνομα , επιτρέποντας στο λειτουργικό σύστημα να αναγνωρίσει το αίτημα και να αναγκάσει τους **απομονωμένους** χώρους εικονικών διευθύνσεων να συγκλίνουν στο ίδιο ακριβώς frame της φυσικής μνήμης RAM.

Υλοποίηση χωρίς semaphores

1. Α. Με ποιο τρόπο και σε ποιο σημείο επιτυγχάνεται ο συγχρονισμός σε αυτή την υλοποίηση;

Στην τρέχουσα υλοποίηση (με τον πλήρη buffer διαστάσεων **y_chars × x_chars**), ο συγχρονισμός επιτυγχάνεται αποκλειστικά στο τέλος της εκτέλεσης των θυγατρικών διεργασιών, μέσα στη συνάρτηση **parent()**, μέσω της κλήσης συστήματος **wait()**.

Δεν απαιτούνται ενδιάμεσοι μηχανισμοί (όπως σημαφόροι) κατά τη φάση του υπολογισμού, διότι επιτυγχάνεται απόλυτος χωρικός διαχωρισμός των δεδομένων. Επειδή η κάθε διεργασία-παιδί γράφει σε αυστηρά διαφορετικές, ανεξάρτητες γραμμές μνήμης (λόγω του βήματος **line += NPROCS**), εξαλείφονται πλήρως οι Race Conditions. Έτσι, ο μοναδικός συγχρονισμός που απαιτείται είναι η επιβεβαίωση (μέσω της **wait**) ότι όλα τα παιδιά έχουν ολοκληρώσει τις εγγραφές τους, ώστε ο γονέας να προχωρήσει στην ασφαλή και σειριακή εκτύπωση ολόκληρου του buffer.

B. Πώς θα επηρεαζόταν το σχήμα συγχρονισμού αν ο buffer είχε διαστάσεις **NPROCS x x_chars**;

Αν ο διαμοιραζόμενος buffer είχε διαστάσεις ίσες με **NPROCS × x_chars**, η χωρητικότητά του θα επαρκούσε για να αποθηκεύσει ακριβώς **μία μόνο γραμμή ανά διεργασία** κάθε φορά (αφού έχουμε **NPROCS** διεργασίες).

Σε αυτή την περίπτωση, το σχήμα συγχρονισμού θα άλλαζε ριζικά και θα μετατρεπόταν σε ένα πρόβλημα **Παραγωγού - Καταναλωτή** με περιορισμένο χώρο. Η υλοποίηση θα επηρεαζόταν ως εξής:

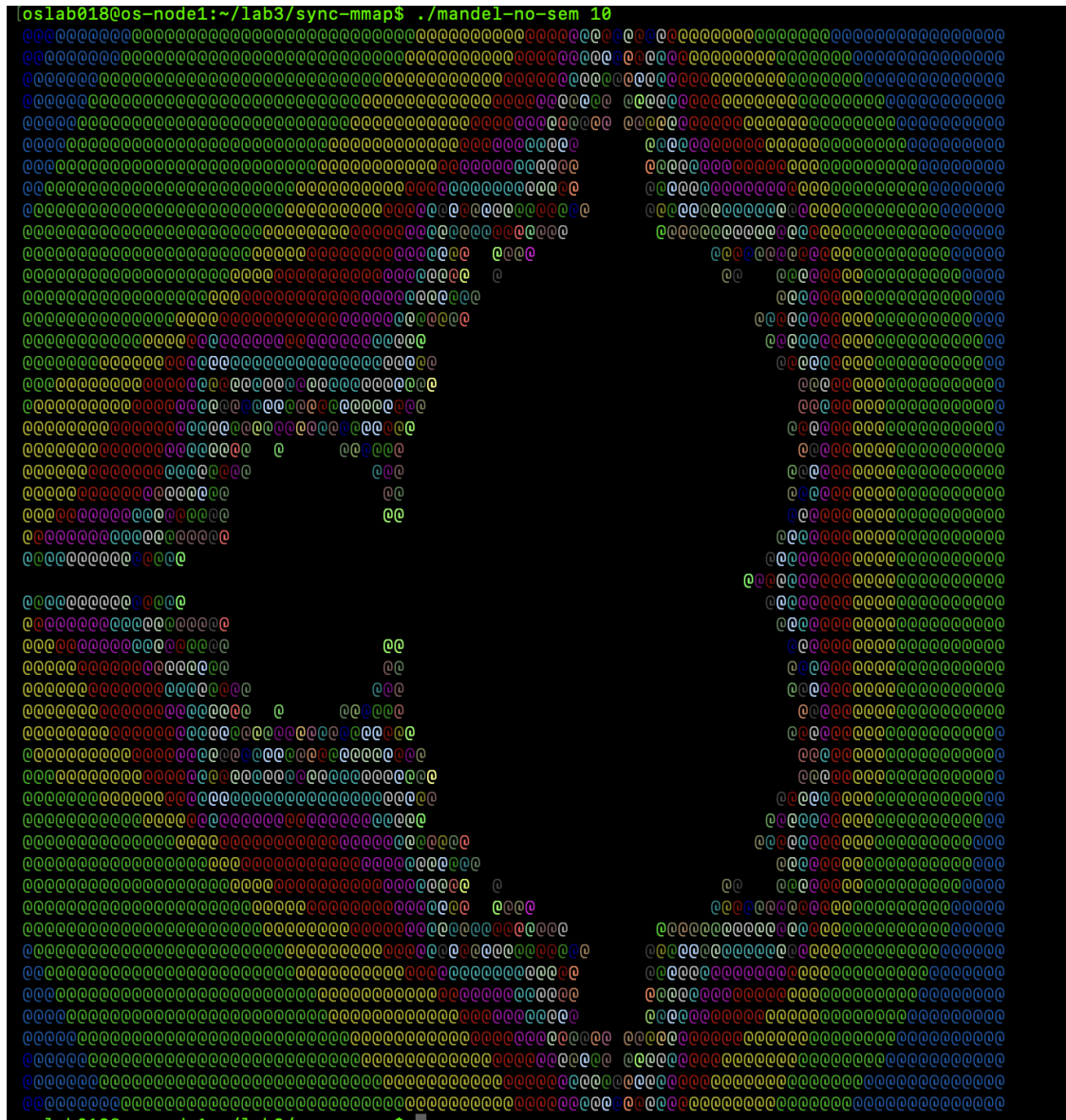
- Ο συγχρονισμός στο τέλος δεν θα επαρκούσε. Θα χρειαζόμασταν υποχρεωτικά **σημαφόρους** για τον συνεχή συντονισμό μεταξύ παιδιών και γονέα σε κάθε γραμμή.
- Ένα παιδί (Παραγωγός) θα υπολόγιζε τη γραμμή του και θα την έγραφε στη μοναδική διαθέσιμη θέση του στον buffer.

- Αυτή η αλλαγή αν και θα απαιτούσε δέσμευση λιγότερου φυσικού χώρου, θα κατέστρεφε την απόδοση του συστήματος. Η συνεχής ανάγκη για εναλλαγές πλαισίου για κάθε επιμέρους γραμμή, θα εισήγαγε τεράστια καθυστέρηση , ακυρώνοντας σε μεγάλο βαθμό τα οφέλη του παράλληλου υπολογισμού.

A. Mε Semaphores:

[illegible]

B. Χωρίς Semaphores:



Source Code (.c):

```
C/C++
/*
 * mandel.c
 *
 *
 * A program to draw the Mandelbrot Set on a 256-color xterm.
 *
 * Dionysis Katsetis < el23005
 *
 * Stefanos Kargas < el23059
 *
 * Synchronized and parallelized version of the original mandel.c program, using
Shared Memory IPC between processes.
 *
 * OS-LAB-3, ECE, NTUA
 *
 * Athens, 2026
 *
 */
```



```

*/
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <signal.h>
#include <sys/wait.h>
#include <ctype.h>
#include "mandel-lib.h"
#define MANDEL_MAX_ITERATION 100000
#define die(msg) \
    do { perror(msg); exit(1); } while (0)
#if defined(SYNC_SEM) ^ defined(SYNC_NO_SEM) == 0
#error You must #define exactly one of SYNC_SEM or SYNC_NO_SEM.
#endif
#if defined(SYNC_SEM)
# define USE_SEM 1
#include <semaphore.h>
sem_t *sems;
#else
int *shared_buffer;
#define USE_SEM 0
#endif
int NPROCS = 3; // we will get them from argv[1] this is default value
/*
 * Output at the terminal is is x_chars wide by y_chars long
 */
int y_chars = 50;
int x_chars = 90;
/*
 * The part of the complex plane to be drawn:
 * upper left corner is (xmin, ymax), lower right corner is (xmax, ymin)
 */
double xmin = -1.8, xmax = 1.0;
double ymin = -1.0, ymax = 1.0;

/*
 * Every character in the final output is
 * xstep x ystep units wide on the complex plane.
 */
double xstep;
double ystep;
/*
 * This function computes a line of output
 * as an array of x_char color values.
 */
void compute_mandel_line(int line, int color_val[])
{
    /*
     * x and y traverse the complex plane.
     */
    double x, y;
    int n;
    int val;
    /* Find out the y value corresponding to this line */

```



```

    y = ymax - ystep * line;
    /* and iterate for all points on this line */
    for (x = xmin, n = 0; n < x_chars; x+= xstep, n++) {
        /* Compute the point's color value */
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);
        if (val > 255)
            val = 255;
        /* And store it in the color_val[] array */
        val = xterm_color(val);
        color_val[n] = val;
    }
}
/*
 * This function outputs an array of x_char color values
 * to a 256-color xterm.
 */
void output_mandel_line(int fd, int color_val[])
{
    int i;

    char point = '@';
    char newline = '\n';
    for (i = 0; i < x_chars; i++) {
        /* Set the current color, then output the point */
        set_xterm_color(fd, color_val[i]);
        if (write(fd, &point, 1) != 1) {
            perror("compute_and_output_mandel_line: write point");
            exit(1);
        }
    }
    /* Now that the line is done, output a newline character */
    if (write(fd, &newline, 1) != 1) {
        perror("compute_and_output_mandel_line: write newline");
        exit(1);
    }
}

void child(void *arg){
    int i = (int)(long)arg; // Process logical ID
    for (int line = i; line < y_chars; line += NPROCS) {
        #if USE_SEM
            int color_val[x_chars];
            compute_mandel_line(line, color_val);
            sem_wait(&sems[i]);
            output_mandel_line(1, color_val);
            sem_post(&sems[(i + 1) % NPROCS]);
        #else
            compute_mandel_line(line, &shared_buffer[line * x_chars]);
        #endif
    }
}

void parent(){
    for (int i = 0; i < NPROCS; i++) {
        if (wait(NULL) < 0) die("wait");
    }
    #if !USE_SEM
        for (int line = 0; line < y_chars; line++) {
            output_mandel_line(1, &shared_buffer[line * x_chars]);
        }
    #endif
}

void sigint_handler(int signum){

```

```

        reset_xterm_color(1);
        _exit(1);
    }
    void *create_shared_memory_area(unsigned int numbytes)
    {
        int pages;
        void *addr;
        if (numbytes == 0) {
            fprintf(stderr, "%s: internal error: called for numbytes == 0\n",
__func__);
            exit(1);
        }
        /*
         * Determine the number of pages needed, round up the requested number of
         * pages
         */
        pages = (numbytes - 1) / sysconf(_SC_PAGE_SIZE) + 1;
        /* Create a shared, anonymous mapping for this number of pages */
        /*
         * CREATION OF A ANONYMOUS MAPPING:
         */
        addr =
mmap(NULL, pages*sysconf(_SC_PAGE_SIZE), PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOU
S, -1, 0);
        if (addr == MAP_FAILED) die("mmap share");
        return addr;
    }
    void destroy_shared_memory_area(void *addr, unsigned int numbytes) {
        int pages;
        if (numbytes == 0) {
            fprintf(stderr, "%s: internal error: called for numbytes == 0\n",
__func__);
            exit(1);
        }

        if (addr == NULL) die("called for null ptr");
        /*
         * Determine the number of pages needed, round up the requested number of
         * pages
         */
        pages = (numbytes - 1) / sysconf(_SC_PAGE_SIZE) + 1;
        if (munmap(addr, pages * sysconf(_SC_PAGE_SIZE)) == -1) {
            perror("destroy_shared_memory_area: munmap failed");
            exit(1);
        }
    }
}
int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "Usage: %s <num_processes>\n", argv[0]);
exit(1); }
    for (int i = 0; argv[1][i] != '\0'; i++) { if (!isdigit(argv[1][i])) {
fprintf(stderr, "Error: '%s' is not a valid positive integer.\n", argv[1]);
exit(1); } }

    struct sigaction sigint_sa;
    sigint_sa.sa_handler = sigint_handler;
    sigset_t mask;
    sigemptyset(&mask);
    sigint_sa.sa_mask = mask;
    sigint_sa.sa_flags = 0;
    int s = sigaction(SIGINT, &sigint_sa, NULL);

```

```

    if (s == -1) die("sigaction");
    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    NPROCS = atoi(argv[1]);
    if(NPROCS <= 0) die("invalid number of processes");
    #if USE_SEM
        sems = (sem_t *)create_shared_memory_area(NPROCS * sizeof(sem_t));
        if (sem_init(&sems[0], 1, 1) < 0) die("sem_init");
        for (int i = 1; i < NPROCS; ++i) {
            if (sem_init(&sems[i], 1, 0) < 0) die("sem_init");
        }
    #else
        shared_buffer = (int *)create_shared_memory_area(y_chars * x_chars *
sizeof(int));
    #endif
    for (int i = 0; i < NPROCS; i++) {
        pid_t pid = fork();
        if (pid < 0) die("fork");
        else if (pid == 0) {
            child((void *) (long)i);
            exit(0);
        }
    }
    parent();
    #if USE_SEM
        for (int i = 0; i < NPROCS; i++) {
            sem_destroy(&sems[i]);
        }
        destroy_shared_memory_area(sems, NPROCS * sizeof(sem_t));
    #else
        destroy_shared_memory_area(shared_buffer, y_chars * x_chars *
sizeof(int));
    #endif
    reset_xterm_color(1);
    return 0;
}

```

Note: Όπως και στην Εργαστηριακή Άσκηση 2, φτιάξαμε έναν πηγαίο κώδικα που με κατάλληλα Preprocessor Directives παράγουμε 2 εκτελέσιμα.

Makefile:

```

None
#
# Makefile for part 2 - mandel.c
#
CC = gcc
CFLAGS = -Wall -O2 -pthread
LIBS =
all: mandel-sem mandel-no-sem
mandel-sem: mandel-lib.o mandel-sem.o
    $(CC) $(CFLAGS) -o mandel-sem mandel-lib.o mandel-sem.o $(LIBS)
mandel-no-sem: mandel-lib.o mandel-no-sem.o
    $(CC) $(CFLAGS) -o mandel-no-sem mandel-lib.o mandel-no-sem.o $(LIBS)
mandel-lib.o: mandel-lib.h mandel-lib.c

```

```

        $(CC) $(CFLAGS) -c -o mandel-lib.o mandel-lib.c
mandel-sem.o: mandel.c
        $(CC) $(CFLAGS) -DSYNC_SEM -c -o mandel-sem.o mandel.c
mandel-no-sem.o: mandel.c
        $(CC) $(CFLAGS) -DSYNC_NO_SEM -c -o mandel-no-sem.o mandel.c
clean:
        rm -f *.s *.o mandel-sem mandel-no-sem

```

Τροποποιήθηκε κατάλληλα ώστε να υποστηρίζει τον μηχανισμό που περιγράψαμε παραπάνω.

Μέρος 3:

Source Code (.c)

Η library utils.{c,h} είναι η ίδια με αυτή της πρώτης εργαστηριακής άσκησης.

```

C/C++
/*
 * This is an upgrade of pfork.c implemented in LAB 1 - Part 3
 * We use IPC with shared memory mechanism
 * We achieve synchronization with semaphores over the shared region of data
 *
 * Stefanos Kargas
 * Dionysis Katsetis
 *
 * OS-LAB-3
 *
 *
 *Athens, 2026
 *
 */
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <signal.h>
#include <semaphore.h>
#include <sys/mman.h>
#include "utils.h"
#define buff_size 1024
#define P 4
struct child_info{
    int id;
    int target;
    off_t start;
    off_t end;

```

```

    char *filename;
};
typedef struct {
    sem_t mutex;
    int total_count;
} shared_data_t;
shared_data_t *shared;
volatile sig_atomic_t active_children=0;
volatile sig_atomic_t sigint_flag = 0;
volatile sig_atomic_t exited_child=0;
void sigint_handler(int signal_num){
    sigint_flag = -1;
}
volatile sig_atomic_t child_exit_code[P];
void sigchld_handler(int signal_num){
    int status;
    while(waitpid(-1,&status,WNOHANG)>0){
        if(exited_child<P) {
            if(WIFEXITED(status)){
                child_exit_code[exited_child] = WEXITSTATUS(status);
            }
            else child_exit_code[exited_child] = -1;
            exited_child++;
        }
        active_children--;
    }
}
void child(struct child_info *child_task){
    int fdr = open(child_task->filename,O_RDONLY);
    check(fdr,"Error at opening the input file\n");
    check(lseek(fdr,child_task->start,SEEK_SET),"Error at lseek\n");
    signal(SIGINT,SIG_IGN); // child has to ignore the SIGINT signal

    int cnt;
    ssize_t rcnt;
    char buff[buff_size];
    off_t current = child_task->start;
    while(current<child_task->end){
        ssize_t bytes_left = buff_size;
        if(bytes_left+current>child_task->end)    bytes_left =
child_task->end-current;
        rcnt = read(fdr,buff,bytes_left);
        if(rcnt==0) break;
        check(rcnt,"Error at reading from input file\n");
        cnt=0;
        for(ssize_t i=0;i<rcnt;++i){
            if(child_task->target==buff[i]) cnt++;
        }
        sem_wait(&shared->mutex);
        shared->total_count += cnt;
        sem_post(&shared->mutex);
        current += rcnt;
        sleep(child_task->id*2+3);
    }
    check(close(fdr),"Error at closing the input file\n");
    _exit(0);
}
int main(int argc,char* argv[]){
    if(argc!=4 || strlen(argv[3])!=1) check(-1,"Invalid input\n");
    char target = argv[3][0];
    //-----Handling the sigint-----//

```

```

    struct sigaction sa_sigint;
    sigset_t sigint_mask;
    sa_sigint.sa_handler = sigint_handler;
    sigemptyset(&sigint_mask);
    sa_sigint.sa_mask = sigint_mask; // OS adds automatically the signal we want to
    handle for blocking in our mymask
    sa_sigint.sa_flags = SA_RESTART; // We dont want other possible syscalls
    running to be blocked while handler is running
    int tell_os_sigint = sigaction(SIGINT,&sa_sigint,NULL);
    check(tell_os_sigint,"Error at sigaction for sigint\n");
    //-----//

    //-----Handling sigchld-----//
    struct sigaction sa_sigchld;
    sigset_t sigchld_mask;
    sa_sigchld.sa_handler = sigchld_handler;
    sigemptyset(&sigchld_mask);
    sa_sigchld.sa_mask = sigchld_mask;
    sa_sigchld.sa_flags = SA_RESTART;
    int tell_os_sigchld = sigaction(SIGCHLD,&sa_sigchld,NULL);
    check(tell_os_sigchld,"Error at sigaction for sigchld\n");
    //-----//

    struct stat stat_buff;
    check(stat(argv[1],&stat_buff),"Error at stat or the input file just does not
    exist in this directory \n");
    off_t size_r = stat_buff.st_size;
    off_t chunk = size_r/P;
    //-----//

    int fdw = open(argv[2],O_CREAT|O_WRONLY|O_TRUNC,S_IRUSR|S_IWUSR);
    check(fdw,"Error at opening/creating output.txt\n");
    shared = mmap(NULL, sizeof(shared_data_t), PROT_READ | PROT_WRITE, MAP_SHARED |
    MAP_ANONYMOUS, -1, 0);
    if (shared == MAP_FAILED) check(-1,"Error at mmap for shared data\n");
    shared->total_count = 0;
    check(sem_init(&shared->mutex,1,1),"sem_init\n");
    for(int j=0;j<P;++j){
        active_children++;
        pid_t p = fork();
        check(p,"Error at fork()\n");
        if(p==0){
            struct child_info child_task;
            child_task.id = j;
            child_task.target = target;
            child_task.start = j*chunk;
            child_task.filename = argv[1];
            if(j==P-1){
                child_task.end = size_r;
            }
            else child_task.end = child_task.start+chunk;
            child(&child_task);
        }
    }

    // Parent process
    sigset_t mask,oldmask;
    sigemptyset(&mask);
    sigaddset(&mask,SIGINT);
    sigaddset(&mask,SIGCHLD);
    sigprocmask(SIG_BLOCK,&mask,&oldmask);

    while(active_children > 0){
        if(sigint_flag == -1 && active_children > 0){

```

```

        sem_wait(&shared->mutex);
        int current_total = shared->total_count;
        sem_post(&shared->mutex);
        char msg[150];
        int n = snprintf(msg, sizeof(msg), "Children active: %d. Target '%c'
found %d times so far.\n", active_children, target, current_total);
        writes(1, msg, n);

        sigint_flag = 0;
    }
    sigsuspend(&oldmask);
}
sigprocmask(SIG_SETMASK, &oldmask, NULL);
for(int i=0; i<P; ++i){
    if(child_exit_code[i] != 0){
        check(-1, "Error, one child returned abnormally\n");
    }
}
char pr[30];
int len = snprintf(pr, sizeof(pr), "The result is : %d \n", shared->total_count);
check(writes(fdw, pr, len), "Error at writing in output file\n");
check(close(fdw), "Error by closing fdw by the parent\n");
sem_destroy(&shared->mutex);
check(munmap(shared, sizeof(shared_data_t)), "munmap\n");
return 0;
}

```

Ανάλυση Κώδικα:

1. Εισαγωγή και Σκοπός της Τροποποίησης

Ο μηχανισμός επικοινωνίας μέσω ανώνυμων αγωγών αντικαταστάθηκε από ένα σχήμα Διαμοιραζόμενης Μνήμης (Shared Memory). Αυτή η αλλαγή επιτρέπει τον άμεσο διαμοιρασμό της κατάστασης της εφαρμογής μεταξύ των διεργασιών παιδιών και του γονέα, δίνοντας τη δυνατότητα παρακολούθησης της προόδου σε πραγματικό χρόνο.

2. Δομή Δεδομένων και Διαμοιραζόμενη Μνήμη

Για την επίτευξη του στόχου, ορίστηκε η δομή **shared_data_t**, η οποία ενθυλακώνει τόσο την κοινή μεταβλητή του μετρητή (**total_count**) όσο και τον POSIX σημαφόρο (**mutex**) που την προστατεύει. Η δέσμευση της μνήμης πραγματοποιήθηκε πριν από τη δημιουργία των διεργασιών, μέσω της κλήσης συστήματος **mmap** με τις σημαίες **MAP_SHARED | MAP_ANONYMOUS**, εξασφαλίζοντας ότι όλες οι διεργασίες (γονέας και παιδιά) έχουν άμεση πρόσβαση στον ίδιο φυσικό χώρο μνήμης, καθώς τα παιδιά κληρονομούν τον Page Table του γονέα χωρίς **cow** για “touch” του διαμοιραζόμενου αυτού frame.

3. Συγχρονισμός και Αμοιβαίος Αποκλεισμός

Επειδή πολλαπλές διεργασίες - παιδιά επιχειρούν να τροποποιήσουν ταυτόχρονα την κοινή μεταβλητή `total_count`, ανακύπτει το πρόβλημα των Συνθηκών Συναγωνισμού (Race Conditions).

- Για την αποφυγή τους, χρησιμοποιήθηκε ο binary **σημαφόρος mutex** (αρχικοποιημένος στο 1 με `pshared = 1`), ο οποίος επιβάλλει αμοιβαίο αποκλεισμό (Mutual Exclusion).
- Μόλις ολοκληρωθεί η σάρωση του chunk (που όπως και στην original μορφή της άσκησης) γίνεται με block by block reading, η διεργασία εισέρχεται στο Κρίσιμο Τμήμα (`sem_wait`), προσθέτει το μερικό άθροισμα στον κεντρικό μετρητή και εξέρχεται (`sem_post`), επαναλαμβάνοντας μέχρι να τελειώσουν τα block του chunk που της έχει δοθεί.

4. Ασύγχρονη Παρακολούθηση μέσω SIGINT

Η χρήση της διαμοιραζόμενης μνήμης επιτρέπει στη γονική διεργασία να γνωρίζει ανά πάσα στιγμή το τρέχον άθροισμα.

- Ο γονέας τίθεται σε κατάσταση αναμονής μέσω της `sigsuspend`, περιμένοντας σήματα ελέγχου, όπως και στην 1η Εργαστηριακή Άσκηση.
- Όταν ο χρήστης αποστέλλει το σήμα SIGINT (μέσω Control+C), ενεργοποιείται ο αντίστοιχος handler, ο οποίος θέτει τη σημαία `sigint_flag`, όπως και στην 1η Εργαστηριακή Άσκηση.
- Κατά την επιστροφή στη ροή ελέγχου, ο **γονέας** διαπιστώνει την αλλαγή της σημαίας, **κλειδώνει με τη σειρά του τον σημαφόρο** (ώστε να μην διαβάσει αλλοιωμένη τιμή ενόσω κάποιο παιδί γράφει), ανακτά την τρέχουσα τιμή του `total_count` και την εκτυπώνει στο τερματικό.

5. Διαχείριση Πόρων

Οι δομές πληροφοριών των παιδιών (`child_task`) δηλώνονται στη στοίβα (`stack`) του εκάστοτε παιδιού αποφεύγοντας την περιττή χρήση της `malloc`. Τέλος, ο γονέας, αφού επιβεβαιώσει τον ομαλό τερματισμό όλων των παιδιών, αποδεσμεύει σωστά τον σημαφόρο (`sem_destroy`) και τη διαμοιραζόμενη μνήμη (`munmap`).

Αποτελέσματα:

```
[oslab018@os-node1:~/lab3/part3$ cat input.txt
Hello World!
[oslab018@os-node1:~/lab3/part3$ ./pfork input.txt output.txt 1
^CChildren active: 4. Target 'l' found 3 times so far.
^CChildren active: 4. Target 'l' found 3 times so far.
^CChildren active: 3. Target 'l' found 3 times so far.
^CChildren active: 2. Target 'l' found 3 times so far.
^CChildren active: 1. Target 'l' found 3 times so far.
^CChildren active: 1. Target 'l' found 3 times so far.
^CChildren active: 1. Target 'l' found 3 times so far.
[oslab018@os-node1:~/lab3/part3$ cat output.txt
The result is : 3
oslab018@os-node1:~/lab3/part3$
```

```
[oslab018@os-node1:~/lab3/part3$ ./pfork input.txt output.txt o
^CChildren active: 4. Target 'o' found 2 times so far.
^CChildren active: 3. Target 'o' found 2 times so far.
^CChildren active: 3. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 2. Target 'o' found 2 times so far.
^CChildren active: 1. Target 'o' found 2 times so far.
^CChildren active: 1. Target 'o' found 2 times so far.
^CChildren active: 1. Target 'o' found 2 times so far.
[oslab018@os-node1:~/lab3/part3$ cat output.txt
The result is : 2
oslab018@os-node1:~/lab3/part3$
```

Makefile

```
None
CC = gcc
CFLAGS = -Wall -Wextra -O2 -pthread
TARGETS = pfork
pfork: pfork_v2.c utils.c
    $(CC) $(CFLAGS) -o pfork pfork_v2.c utils.c
clean:
    rm -f $(TARGETS) *.o output.txt
```